

# JavaScript

Damian Nicolalde-R.  
danicolalde@puce.edu.ec

# Temas:

- Introducción a JavaScript
- Funciones
- Objetos
- **Objetos estándar**
- Eventos
- Depuración
- API

# Objetos estándar

- Los objetos son como envoltorios de código y JavaScript se aprovecha de esta característica extensamente. De hecho, casi todo en JavaScript es un objeto. Por ejemplo, los números y las cadenas de caracteres que asignamos a las variables se convierten automáticamente en objetos por el intérprete JavaScript. Cada vez que asignamos un nuevo valor a una variable, en realidad estamos asignando un objeto que contiene ese valor.
- Debido a que los valores que almacenamos en variables son de tipos diferentes, existen diferentes tipos de objetos disponibles para representarlos. Por ejemplo, si el valor es una cadena de caracteres, el objeto creado para almacenarlo es del tipo String. Cuando asignamos un texto a una variable, JavaScript crea un objeto String, almacena la cadena de caracteres en el objeto y asigna ese objeto a la variable. Si queremos, podemos crear estos objetos directamente usando sus constructores. Existen diferentes constructores disponibles dependiendo del tipo de valor que queremos almacenar. Los siguientes son los más usados.

# Objetos estándar

- **Number(valor)**—Este constructor crea objetos para almacenar valores numéricos. Acepta números y también cadenas de caracteres con números. Si el valor especificado por el atributo no se puede convertir en un número, el constructor devuelve el valor NaN (No es un Número).
- **String(valor)**—Este constructor crea objetos para almacenar cadenas de caracteres. Acepta una cadena de caracteres o cualquier valor que pueda convertirse en una cadena de caracteres, incluidos números.
- **Boolean(valor)**—Este constructor crea objetos para almacenar valores booleanos. Acepta los valores true y false. Si el valor se omite o es igual a 0, NaN, null, undefined, o una cadena de caracteres vacía, el valor almacenado en el objeto es false, en caso contrario es true.
- **Array(valores)**—Este constructor crea objetos para almacenar arrays. Si se proveen múltiples valores, el constructor crea un array con esos valores, pero si solo se provee un valor, y ese valor es un número entero, el constructor crea un array con la cantidad de elementos que indica el valor del atributo y almacena el valor undefined en cada índice.

# Objetos estándar

- Estos constructores trabajan con el operador new para crear nuevos objetos. El siguiente ejemplo almacena un número (ver **Listado 6-83**).

```
var valor = new Number(5);  
alert(valor); // 5
```

- La ventaja de usar constructores es que pueden procesar diferentes tipos de valores. Por ejemplo, podemos obtener un número a partir de una cadena de caracteres que contiene un número (ver **Listado 6-84**).

```
var valor = new Number("5");  
alert(valor); // 5
```

# Objetos estándar

- La cadena de caracteres que provee al constructor `Number()` en el Listado 6-84 se convierte en un valor numérico y se almacena en un objeto `Number`. Debido a que JavaScript se encarga de convertir estos objetos en valores primitivos y viceversa, podemos realizar operaciones aritméticas sobre el valor almacenado en el objeto, como lo hacemos con cualquier otro valor numérico (ver **Listado 6-85**).

```
var valor = new Number("5");  
var total = valor * 35;  
alert(total); // 175
```

# Objetos estándar

- El constructor `Array()` se comporta de un modo diferente. Si especificamos varios valores, el array se crea como si lo hubiéramos declarado con corchetes.(ver **Listado 6-86**).

```
var lista = new Array(12, 35, 57, 8);  
alert(lista); // 12,35,57,8
```

- Por otro lado, si especificamos un único valor y ese valor es un número entero, el constructor lo utiliza para determinar el tamaño del array, crea un array con esa cantidad de elementos y asigna el valor `undefined` a cada uno de ellos.

```
var lista = new Array(2);  
alert(lista[0] + " - " + lista[1]); // undefined - undefined
```

# Objetos String

Convertir valores en objetos permite al lenguaje ofrecer la funcionalidad necesaria para manipular esos valores. Las siguientes son las propiedades y los métodos más usados de los objetos String.

- **Length**—Esta propiedad devuelve un entero que representa la cantidad de caracteres en la cadena.
- **toLowerCase()**—Este método convierte los caracteres a letras minúsculas.
- **toUpperCase()**—Este método convierte los caracteres a letras mayúsculas.
- **Trim()**—Este método elimina espacios en blanco a ambos lados de la cadena de caracteres. JavaScript también incluye los métodos `trimLeft()` y `trimRight()` para limpiar la cadena de caracteres en un lado específico (izquierda o derecha).
- **Substr(comienzo, extensión)**—Este método devuelve una nueva cadena de caracteres con caracteres extraídos de la cadena original. El atributo **comienzo** indica la posición del primer carácter a leer, y el atributo **extensión** determina cuántos caracteres queremos incluir. Si no se especifica la extensión, el método devuelve todos los caracteres hasta el final de la cadena.
- **Substring(comienzo, final)**—Este método devuelve una nueva cadena de caracteres con caracteres extraídos de la cadena original. Los atributos **comienzo** y **final** son números enteros que determinan las posiciones del primer y último carácter a incluir.

# Objetos String

- **Includes**(buscar, posición)—Este método busca el valor del atributo **buscar** dentro de la cadena y devuelve true o false de acuerdo con el resultado. El atributo **buscar** es el texto que queremos buscar, y el atributo **posición** determina el índice en el que queremos comenzar la búsqueda. Si el atributo **posición** no se especifica, la búsqueda comienza desde el inicio de la cadena.
- **indexOf**(valor, posición)—Este método devuelve el índice en el que el texto especificado por el atributo **valor** se encuentra por primera vez. El atributo **posición** determina el índice en el que queremos comenzar la búsqueda. Si el atributo **posición** no se especifica, la búsqueda comienza desde el inicio de la cadena. El método devuelve el valor -1 si ninguna coincidencia es encontrada.
- **lastIndexOf**(valor, posición)—Este método devuelve el índice en el que se encuentra por primera vez el texto especificado por el atributo **valor**. A diferencia de `indexOf()`, este método realiza una búsqueda hacia atrás, desde el final de la cadena. El atributo **posición** determina el índice en el que queremos comenzar la búsqueda. Si no se especifica el atributo **posición**, la búsqueda comienza desde el final de la cadena. El método devuelve el valor -1 si no se encuentra ninguna coincidencia.
- **Replace**(expresión, reemplazo)—Este método reemplaza la parte de la cadena de caracteres que coincide con el valor del atributo **expresión** por el texto especificado por el atributo **reemplazo**. El atributo **expresión** se puede especificar como una cadena de caracteres o como una expresión regular para buscar un texto con un formato particular.

# Objetos String

- Las cadenas de caracteres se almacenan como arrays de caracteres, de modo que podemos acceder a cada carácter usando corchetes, como lo hacemos con cualquier otro array. JavaScript incluye la propiedad `length` para contar la cantidad de caracteres en la cadena (ver **Listado 6-88**).

```
var texto = "Hola Mundo";  
var mensaje = "El texto tiene " + texto.length + " caracteres";  
alert(mensaje); // "El texto tiene 10 caracteres"
```

- Debido a que las cadenas de caracteres se almacenan como arrays, podemos iterar sobre los caracteres con un bucle. En el siguiente ejemplo, agregamos un espacio entre las letras de un texto (ver **Listado 6-89**).

```
var texto = "Hola Mundo";  
var mensaje = "";  
for (var f = 0; f < texto.length; f++) {  
    mensaje += texto[f] + " ";  
}  
alert(mensaje); // "H o l a   M u n d o "
```

# Objetos String

- Este ejemplo agrega un espacio después de cada carácter en la cadena, lo cual significa que el texto final termina con un espacio en blanco. Los objetos String ofrecen el método `trim()` para eliminar estos espacios no deseados.(ver *Listado 6-90*).

```
var texto = "Hola Mundo";
var mensaje = "";
for (var f = 0; f < texto.length; f++) {
    mensaje += texto[f] + " ";
}
mensaje = mensaje.trim();
alert(mensaje);    // "H o l a   M u n d o"
```

# Objetos String

- La posibilidad de acceder a cada carácter en una cadena nos permite lograr efectos interesantes. Solo tenemos que detectar la posición del carácter que queremos manipular y realizar los cambios que deseamos. El siguiente ejemplo agrega puntos entre las letras de cada palabra, pero no entre las palabras (ver **Listado 6-91**).

```
var texto = "Hola Mundo";
var mensaje = "";
var anterior = "";

for (var f = 0; f < texto.length; f++) {
    if (mensaje != "") {
        anterior = texto[f - 1];
        if (anterior != " " && texto[f] != " ") {
            mensaje += ".";
        }
    }
    mensaje += texto[f];
}
alert(mensaje); // "H.o.l.a M.u.n.d.o"
```

# Objetos String

- Debido a la complejidad que pueden alcanzar algunos de los procesos realizados con cadenas de caracteres, JavaScript incluye varios métodos para simplificar nuestro trabajo. Por ejemplo, podemos reemplazar todos los caracteres en una cadena con letras mayúsculas con solo llamar al método `toUpperCase()`. (ver Listado 6-92).

```
var texto = "Hola Mundo";  
var mensaje = texto.toUpperCase();  
alert(mensaje);    // "HOLA MUNDO"
```

# Objetos String

- A veces no necesitamos trabajar con todo el texto, sino solo con algunas palabras o caracteres. Los objetos String incluyen los métodos `substr()` y `substring()` para copiar un trozo de texto desde una cadena. El método `substr()` copia el grupo de caracteres que comienza en el índice especificado por el primer atributo. También se puede especificar un segundo atributo para determinar cuántos caracteres queremos incluir desde la posición inicial.(ver Listado 6-93).

```
var texto = "Hola Mundo";  
var palabra = texto.substr(0, 4);  
alert(palabra);    // "Hola"
```

# Objetos String

- El método `substr()` del Listado 6-93 copia un total de cuatro caracteres comenzando con el carácter en el índice 0. Como resultado, obtenemos la cadena “Hola”. Si no especificamos el número de caracteres a incluir, el método devuelve todos los caracteres hasta que llega al final de la cadena. (ver Listado 6-94).

```
var texto = "Hola Mundo";  
var palabra = texto.substr(5);  
alert(palabra);    // "Mundo"
```

# Objetos String

- El método `substr()` también puede tomar valores negativos. Cuando se especifica un índice negativo, el método cuenta de atrás hacia adelante. El siguiente código copia los mismos caracteres que el ejemplo anterior.(ver Listado 6-95).

```
var texto = "Hola Mundo";  
var palabra = texto.substr(-5);  
alert(palabra); // "Mundo"
```

# Objetos String

- El método `substring()` trabaja de una forma diferente al método `substr()`. Este método toma dos valores para determinar los caracteres primero y último que queremos copiar, pero no incluye el último carácter. (ver Listado 6-96).

```
var texto = "Hola Mundo";  
var palabra = texto.substring(5, 7);  
alert(palabra); // "Mu"
```

- El método `substring()` del Listado 6-96 copia los caracteres desde el índice 5 al 7 de la cadena (el carácter en el último índice no se incluye). Estas son las posiciones en las que se encuentran los caracteres "Mu" y, por lo tanto, este es el valor que obtenemos en respuesta.

# Objetos String

- Si lo que necesitamos es extraer las palabras de una cadena, podemos usar el método `split()`. Este método corta la cadena en partes más pequeñas y devuelve un array con estos valores. El método requiere un valor con el carácter que queremos usar como separador, por lo que si usamos un espacio, podemos dividir la cadena en palabras (ver Listado 6-97).

```
var texto = "Hola Mundo";  
var palabras = texto.split(" ");  
alert(palabras[0] + " / " + palabras[1]); // "Hola / Mundo"
```

# Objetos String

- A veces no sabemos dónde se encuentran los caracteres que queremos modificar o si la cadena incluye esos caracteres. Existen varios métodos disponibles en los objetos String para hacer una búsqueda. El que usemos depende de lo que queremos lograr. Por ejemplo, los métodos `startsWith()` y `endsWith()` buscan un texto al comienzo o al final de la cadena, y devuelven `true` si se encuentra el texto.(ver Listado 6-98).

```
var texto = "Hola Mundo";  
if (texto.startsWith("Ho")) {  
    alert("El texto comienza con 'Ho'");  
}
```

- Debido a que estos métodos devuelven valores booleanos, podemos usarlos para establecer la condición de una instrucción `if`. En el código del Listado 6-98, buscamos el texto "Ho" al comienzo de la variable `texto` y mostramos un mensaje en caso de éxito. En este ejemplo, se encuentra el texto, el método devuelve `true` y, por lo tanto, el mensaje se muestra en pantalla.

# Objetos String

- Si el texto que estamos buscando puede encontrarse en cualquier parte de la cadena, no solo al comienzo o al final, podemos usar el método `includes()`. Al igual que los métodos anteriores, el método `includes()` busca un texto y devuelve `true` en caso de éxito, pero la búsqueda se realiza en toda la cadena (ver Listado 6-99).

```
var texto = "Hola Mundo";  
if (texto.includes("l")) {  
    alert("El texto incluye la letra L");  
}
```

# Objetos String

- Hasta el momento, hemos comprobado si uno o más caracteres se encuentran dentro de una cadena, pero a veces necesitamos saber dónde se han encontrado esos caracteres. Existen dos métodos para este propósito: `indexOf()` y `lastIndexOf()`. Ambos métodos devuelven el índice donde se encuentra la primera coincidencia, pero el método `indexOf()` comienza la búsqueda desde el inicio de la cadena y el método `lastIndexOf()` lo hace desde el final.(ver Listado 6-100).

```
var texto = "Hola Mundo";  
var 288áximo = texto.indexOf("Mundo");  
alert("La palabra comienza en el índice " + 288áximo); // 5
```

# Objetos String

- Una vez que conocemos la posición de los caracteres que estamos buscando, podríamos crear un pequeño código que reemplace esos caracteres por otros diferentes, pero esto no es necesario. Los objetos String incluyen un método llamado `replace()` con este propósito. Este es un método complejo que puede tomar múltiples valores y trabajar no solo con cadenas de caracteres, sino con expresiones regulares, pero también podemos usarlo con textos breves o palabras. El siguiente ejemplo reemplaza la palabra “Mundo” por la palabra “Planeta”.(ver Listado 6-101).

```
var texto = "Hola Mundo";  
var textonuevo = texto.replace("Mundo", "Planeta");  
alert(textonuevo); // "Hola Planeta"
```

# Objetos Array

Como ya mencionamos, en JavaScript los arrays también son objetos e incluyen propiedades y métodos para manipular sus valores. Los siguientes son los más usados.

- **Length**—Esta propiedad devuelve un número entero que representa la cantidad de valores en el array.
- **Push(valores)**—Este método agrega uno o más valores al final del array y devuelve la nueva extensión del array. También contamos con un método similar llamado **unshift()**, que agrega los valores al comienzo del array.
- **Pop()**—Este método elimina el último valor del array y lo devuelve. También contamos con un método similar llamado **shift()**, que elimina el primer valor del array.
- **Concat(array)**—Este método concatena el array con el array especificado por el atributo y devuelve un nuevo array con el resultado. Los arrays originales no se modifican.
- **Splice(índice, remover, valores)**—Este método agrega o elimina valores de un array y devuelve un nuevo array con los elementos eliminados. El atributo **índice** indica el índice en el que vamos a introducir las modificaciones, el atributo **remover** es el número de valores que queremos eliminar desde el índice, y el atributo **valores** es la lista de valores separados por coma que queremos agregar al array desde el índice. Si solo queremos agregar valores, el atributo **remover** se puede declarar con el valor 0, y si solo queremos eliminar valores, podemos ignorar el último atributo.

# Objetos Array

- **Slice**(comienzo, final)—Este método copia los valores en las posiciones indicadas por los atributos dentro de un nuevo array. Los atributos **comienzo** y **final** indican los índices del primer y último valor a copiar. El último valor no se incluye en el nuevo array.
- **indexOf**(valor, posición)—Este método devuelve el índice en el que se encuentra por primera vez el valor especificado por el atributo **valor**. El atributo **posición** determina el índice en el que queremos comenzar la búsqueda. Si el atributo **posición** no se especifica, la búsqueda comienza desde el inicio del array. El método devuelve el valor -1 si no se encuentra ninguna coincidencia.
- **lastIndexOf**(valor, posición)—Este método devuelve el índice en el que el valor especificado por el atributo **valor** se encuentra por primera vez. A diferencia de **indexOf()**, este método realiza una búsqueda de atrás hacia adelante. El atributo **posición** determina el índice en el que queremos comenzar la búsqueda. Si no se especifica el atributo **posición**, la búsqueda comienza desde el final del array. El método devuelve el valor -1 si no se encuentra ninguna coincidencia.
- **Filter**(función)—Este método envía los valores del array a una función uno por uno y devuelve un nuevo array con todos los valores que aprueba la función. El atributo **función** es una referencia a una función o una función anónima a cargo de validar los valores. La función recibe tres valores: el valor a evaluar, su índice y una referencia al array. Después de procesar el valor, la función debe devolver un valor booleano para indicar si es válido o no.

# Objetos Array

- **every(función)**—Este método envía los valores del array a una función uno por uno y devuelve true cuando la función aprueba todos los valores. El atributo **función** es una referencia a una función o una función anónima a cargo de evaluar los valores. La función recibe tres valores: el valor a evaluar, su índice, y una referencia al array. Después de procesar el valor, la función debe devolver un valor booleano indicando si es válido o no. También contamos con un método similar llamado **some()** que devuelve true si la función aprueba al menos uno de los valores.
- **Join(separador)**—Este método crea una cadena de caracteres con todos los valores en el array. El atributo **separador** especifica el carácter o la cadena de caracteres que queremos incluir entre los valores.
- **Reverse()**—Este método invierte el orden de los valores en el array.
- **Sort(función)**—Este método ordena los valores en el array. El atributo **función** es una referencia a una función o una función anónima a cargo de comparar los valores. La función recibe dos valores desde el array y debe devolver un valor booleano indicando su orden. Si no se especifica el atributo, el método ordena los elementos alfabéticamente y en orden ascendente.
- **Map(función)**—Este método envía los valores del array a una función uno por uno y crea un nuevo array con los valores que devuelve la función. El atributo **función** es una referencia a una función o una función anónima a cargo de procesar los valores. La función recibe tres valores: el valor a procesar, su índice y una referencia al array.

# Objetos Array

- (Ver Listado 6-102)

```
var lista = [12, 5, 80, 34];  
alert(lista.length); // 4
```

- Usando esta propiedad, podemos iterar sobre el array con un bucle for, como hemos hecho anteriormente con cadenas de caracteres. El valor que devuelve la propiedad se usa para definir la condición del bucle (ver Listado 6-103).

```
var lista = [12, 5, 80, 34];  
var total = 0;  
for (var f = 0; f < lista.length; f++) {  
    total += lista[f];  
}  
alert("El total es: " + total); // "El total es: 131"
```

# Objetos Array

- Aunque podemos iterar sobre el array para leer y procesar los valores uno por uno, los objetos Array ofrecen otros métodos para acceder a ellos. Por ejemplo, si queremos procesar solo algunos de los valores en el array, podemos obtener una copia con el método `slice()`. (Ver Listado 6-104)

```
var lista = [12, 5, 80, 34];  
var listanueva = lista.slice(1, 3);  
alert(listanueva); // 5,80
```

- El método `slice()` devuelve un nuevo array con los valores entre el índice especificado por el primer atributo y el valor en el índice que se encuentra antes del especificado por el segundo atributo.

# Objetos Array

- Si queremos examinar los valores antes de incluirlos en el nuevo array, tenemos que usar un filtro. Con este propósito, los objetos Array ofrecen el método `filter()`. Este método envía cada valor a la función y los incluye en el nuevo array solo cuando la función devuelve `true`. En el siguiente ejemplo, devolvemos `true` cuando el valor es menor o igual que 50. En consecuencia, el nuevo array contiene todos los valores del array original excepto el valor 80. (Ver Listado 6-105)

```
var lista = [12, 5, 80, 34];
var listanueva = lista.filter(function(valor) {
    if (valor <= 50) {
        return true;
    } else {
        return false;
    }
});
alert(listanueva); // 12, 5, 34
```

# Objetos Array

- En la función provista para el método filter() del Listado 6-105, comparamos el valor del array con el número 50 y devolvemos true o false dependiendo de la condición, pero como las condiciones ya producen un valor booleano cuando se evalúan, podemos devolver la condición misma y simplificar el código.(Ver Listado 6-106)

```
var lista = [12, 5, 80, 34];  
var listanueva = lista.filter(function(valor) {  
    return valor <= 50;  
});  
alert(listanueva);    // 12, 5, 34
```

# Objetos Array

- Parecidos a `filter()` son los métodos `every()` y `some()`. Estos métodos evalúan los valores con una función, pero en lugar de devolver un array con los valores validados por la función, devuelven los valores `true` o `false`. El método `every()` devuelve `true` si se validan todos los valores, y el método `some()` devuelve `true` si se valida al menos uno de los valores. El siguiente ejemplo usa la función `every()` para comprobar que todos los valores del array son menores o iguales a 100.(Ver Listado 6-107)

```
var lista = [12, 5, 80, 34];
var valido = lista.every(function(valor) {
    return valor <= 100;
});
if (valido) {
    alert("Los valores no son mayores que 100");
}
```

# Objetos Array

- Si lo que queremos es incluir los valores de una array dentro de una cadena de caracteres para mostrarlos al usuario, podemos llamar al método `join()`. Este método crea una cadena de caracteres con los valores del array separados por un carácter o una cadena de caracteres. El siguiente ejemplo crea una cadena de caracteres con un guion entre los valores.(Ver Listado 6-108)

```
var lista = [12, 5, 80, 34];  
var mensaje = lista.join("-");  
alert(mensaje);    // "12-5-80-34"
```

# Objetos Array

- Otra manera de acceder a los valores de un array es con los métodos `indexOf()` y `lastIndexOf()`. Estos métodos buscan un valor y devuelven el índice de la primera coincidencia encontrada. El método `indexOf()` comienza la búsqueda desde el inicio del array y el método `lastIndexOf()` lo hace desde el final. (Ver Listado 6-109)

```
var lista = [12, 5, 80, 34, 5];  
var 292áximo = lista.indexOf(5);  
alert("El valor " + lista[292áximo] + " se encuentra en el índice " +  
292áximo);
```

- El código del Listado 6-109 busca el valor 5 en el array y devuelve el índice 1. Esto se debe a que el método solo devuelve el índice del primer valor que coincide con la búsqueda.

# Objetos Array

- Si queremos encontrar todos los valores que coinciden con la búsqueda, tenemos que crear un bucle para realizar múltiples búsquedas, como muestra el siguiente ejemplo. (Ver Listado 6-110)

```
var lista = [12, 5, 80, 34, 5];
var buscar = 5;
var ultimo = 0;
var contador = 0;
while (ultimo >= 0) {
    var ultimo = lista.indexOf(5, ultimo);
    if (ultimo != -1) {
        ultimo++;
        contador++;
    }
}
alert("Hay un total de " + contador + " números " + buscar);
```

# Objetos Array

- El método `indexOf()` puede tomar un segundo atributo que especifica la ubicación desde la que queremos comenzar la búsqueda. Usando este atributo, podemos indicar al método que no busque en los índices donde ya hemos encontrado un valor. En el ejemplo del Listado 6-110, usamos la variable `ultimo` para este propósito. Esta variable almacena el índice del último valor encontrado por el método `indexOf()`. Al comienzo, la variable se inicializa con el valor 0, lo cual significa que el método comenzará la búsqueda desde el índice 0 del array, pero después de realizar la primera búsqueda, la variable se actualiza con el índice que devuelve el método `indexOf()`, desplazando el punto de partida de la próxima búsqueda a una nueva ubicación. El bucle sigue funcionando hasta que el método `indexOf()` devuelve el valor -1 (no se ha encontrado ninguna coincidencia).
- Además de establecer la nueva ubicación con la variable `ultimo`, el bucle también incrementa el valor de la variable `contador` para contar el número de valores encontrados. Cuando finaliza el proceso, el código muestra un mensaje en la pantalla para comunicar esta información al usuario (“Hay un total de 2 números 5”).
- Hasta el momento, hemos trabajado siempre con el mismo array, pero los arrays se pueden ampliar o reducir. Para agregar valores a un array, todo lo que tenemos que hacer es llamar a los métodos `push()` o `unshift()`. El método `push()` agrega el valor al final del array y el método `unshift()` lo agrega al comienzo.

# Objetos Array

- (Ver Listado 6-111)

```
var lista = [12, 5, 80, 34];  
lista.push(100);  
alert(lista);    // 12,5,80,34,100
```

- Para agregar múltiples valores, tenemos que especificarlos separados por comas. (Ver Listado 6-112)

```
var lista = [12, 5, 80, 34];  
lista.push(100, 200, 300);  
alert(lista);    // 12,5,80,34,100,200,300
```

# Objetos Array

- Otra manera de agregar múltiples valores a un array es con el método `concat()`. Este método concatena dos arrays y devuelve un nuevo array con el resultado (los arrays originales no se modifican).(Ver Listado 6-113)

```
var lista = [12, 5, 80, 34];  
var listanueva = lista.concat([100, 200, 300]);  
alert(listanueva);    // 12,5,80,34,100,200,300
```

# Objetos Array

- Eliminar valores es también fácil de hacer con los métodos `pop()` y `shift()`. El método `pop()` elimina el valor al final del array y el método `shift()` lo elimina al inicio. (Ver Listado 6-114)

```
var lista = [12, 5, 80, 34];  
lista.pop();  
alert(lista); // 12,5,80
```

- Los métodos anteriores agregan o eliminan valores al comienzo o al final del array. Si queremos más flexibilidad, podemos usar el método `splice()`. Este método agrega o elimina valores desde cualquier ubicación del array. El primer atributo del método `splice()` especifica el índice donde queremos comenzar a eliminar valores y el segundo atributo determina cuántos elementos queremos eliminar. Por ejemplo, el siguiente código elimina 2 valores desde el índice 1. (Ver Listado 6-115)

```
var lista = [12, 5, 80, 34];  
var removidos = lista.splice(1, 2);  
alert("Valores restantes: " + lista); // 12,34  
alert("Valores removidos: " + removidos); // 5,80
```

# Objetos Array

- Con este método, también podemos agregar nuevos valores a un array en posiciones específicas. Los valores se deben especificar después de los dos primeros atributos separados por comas. El siguiente ejemplo agrega los valores 24, 25, y 26 en el índice 2. Como no se va a eliminar ningún valor, declaramos el valor 0 para el segundo atributo.(Ver Listado 6-116)

```
var lista = [12, 5, 80, 34];  
lista.splice(2, 0, 24, 25, 26);  
alert(lista);    // 12,5,24,25,26,80,34
```

# Objetos Array

- Los objetos Array también incluyen métodos para ordenar los valores en el array. Por ejemplo, el método `reverse()` invierte el orden de los valores en el array. (Ver Listado 6-117)

```
var lista = [12, 5, 80, 34];  
lista.reverse();  
alert(lista);    // 34,80,5,12
```

- Un mejor método para ordenar arrays es `sort()`. Este método puede tomar una función para decidir el orden en el que se ubicarán los valores. Si no especificamos ninguna función, el método ordena el array en orden alfabético y ascendente (Ver Listado 6-118).

```
var lista = [12, 5, 80, 34];  
lista.sort();  
alert(lista);    // 12,34,5,80
```

# Objetos Array

- Los valores en este ejemplo se ordenan alfabéticamente, no de forma numérica. Si queremos que el método considere el orden numérico o lograr una organización diferente, tenemos que facilitar una función. La función recibe dos valores y debe devolver un valor booleano para indicar su orden. Por ejemplo, para ordenar los valores en orden ascendente, tenemos que devolver true si el primer valor es mayor que el segundo valor o false en caso contrario. (Ver Listado 6-119)

```
var lista = [12, 5, 80, 34];  
lista.sort(function(valor1, valor2) {  
    return valor1 > valor2;  
});  
alert(lista);    // 5,12,34,80
```

# Objetos Array

- Un método interesante que se incluye en los objetos Array es `map()`. Con este método podemos procesar los valores del array uno por uno y crear un nuevo array con los resultados. Al igual que otros métodos, este método envía los valores a una función, pero en lugar de un valor booleano, la función debe devolver el valor que queremos almacenar en el nuevo array. Por ejemplo, el siguiente código multiplica cada valor por 2 y devuelve los resultados. (Ver Listado 6-120)

```
var lista = [12, 5, 80, 34];  
var listanueva = lista.map(function(valor) {  
    return valor * 2;  
});  
alert(listanueva);    // 24,10,160,68
```

# Objetos Date

El objeto Date almacena una fecha y se encarga de mantener los valores dentro de sus límites. La fecha en estos objetos se almacena en milisegundos, lo cual nos permite realizar operaciones entre fechas, calcular intervalos, etc. JavaScript ofrece el siguiente constructor para crear estos objetos.

- **Date(valor)**—Este constructor crea un valor en milisegundos para representar una fecha basada en los valores provistos por el atributo. El atributo valor se puede declarar como una cadena de caracteres o como los componentes de una fecha separados por comas, en este orden: año, mes, día, horas, minutos, segundos y milisegundos. Si no especificamos ningún valor, el constructor crea un objeto con la fecha actual del sistema.

La fecha almacenada en estos objetos se representa con un valor en milisegundos calculado desde el 1 de enero del año 1970.

# Objetos Date

Debido a que este valor no resulta familiar para los usuarios, los objetos Date ofrecen los siguientes métodos para obtener los componentes de la fecha, como el año o el mes.

- **getFullYear()**—Este método devuelve un número entero que representa el año (un valor de 4 dígitos).
- **getMonth()**—Este método devuelve un número entero que representa el mes (un valor de 0 a 11).
- **getDate()**—Este método devuelve un número entero que representa el día del mes (un valor de 1 a 31).
- **getDay()**—Este método devuelve un número entero que representa el día de la semana (un valor de 0 a 6).
- **getHours()**—Este método devuelve un número que representa las horas (un valor de 0 a 23).
- **getMinutes()**—Este método devuelve un número entero que representa los minutos (un valor de 0 a 59).
- **getSeconds()**—Este método devuelve un número entero que representa los segundos (un valor de 0 a 59).
- **getMilliseconds()**—Este método devuelve un número entero que representa los milisegundos (un valor de 0 a 999).
- **getTime()**—Este método devuelve un número entero en milisegundos que representa el intervalo desde el 1 de enero de 1970 hasta la fecha.

# Objetos Date

Los objetos Date también incluyen métodos para modificar los componentes de la fecha.

- **setFullYear(año)**—Este método especifica el año (un valor de 4 dígitos). También puede recibir valores para especificar el mes y el día.
- **setMonth(mes)**—Este método especifica el mes (un valor de 0 a 11). También puede recibir valores para especificar el día.
- **setDate(día)**—Este método especifica el día (un valor de 1 a 31).
- **setHours(horas)**—Este método especifica la hora (un valor de 0 a 23). También puede recibir valores para especificar los minutos y segundos.
- **setMinutes(minutos)**—Este método especifica los minutos (un valor de 0 a 59). También puede recibir un valor para especificar los segundos.
- **setSeconds(segundos)**—Este método especifica los segundos (un valor de 0 a 59). También puede recibir un valor para especificar los milisegundos.
- **setMilliseconds(milisegundos)**—Este método especifica los milisegundos (un valor de 0 a 999).

# Objetos Date

Los objetos Date también ofrecen métodos para convertir una fecha en una cadena de caracteres.

- **toString()**—Este método convierte una fecha en una cadena de caracteres. El valor que devuelve se expresa en inglés americano y con el formato “Wed Jan 04 2017 22:32:48 GMT-0500 (EST)”.
- **toString()**—Este método convierte una fecha en una cadena de caracteres, pero solo devuelve la parte de la fecha, no la hora. El valor se expresa en inglés americano y con el formato “Wed Jan 04 2017”.
- **toString()**—Este método convierte una fecha en una cadena de caracteres, pero solo devuelve la hora. El valor se expresa en inglés americano y con el formato “23:21:55 GMT-0500 (EST)”.

# Objetos Date

Cada vez que necesitamos una fecha, simplemente tenemos que crear un nuevo objeto con el constructor `Date()`. Si no especificamos una fecha, el constructor crea el objeto `Date` con la fecha actual del sistema (ver **Listado 6-121**).

```
var fecha = new Date();  
alert(fecha); // "Wed Jan 04 2017 20:51:17 GMT-0500 (EST)"
```

La fecha se puede declarar con una cadena de caracteres que contiene una fecha expresada en un formato comprensible para las personas. El constructor se encarga de convertir el texto en un valor en milisegundos (ver **Listado 6-122**).

```
var fecha = new Date("January 20 2017");  
alert(fecha); // "Fri Jan 20 2017 00:00:00 GMT-0500 (EST)"
```

# Objetos Date

- Debido a que los navegadores interpretan el texto de diferentes maneras, en lugar de una cadena de caracteres es recomendable crear la fecha declarando los valores de los componentes separados por comas. El siguiente ejemplo crea un objeto Date con la fecha 2017/02/15 12:35.(ver *Listado 6-123*).

```
var fecha = new Date(2017, 1, 15, 12, 35, 0);  
alert(fecha); // "Wed Feb 15 2017 12:35:00 GMT-0500 (EST)"
```

- Los valores se deben declarar en el siguiente orden: año, mes, día, hora, minutos y segundos. Todos los valores son iguales a los que acostumbramos a usar para definir una fecha, pero el mes se debe representar con un número de 0 a 11, y por este motivo hemos tenido que declarar el valor 1 en el constructor para representar el mes de febrero.

# Objetos Date

- Debido a que las fechas se almacenan en milisegundos, cada vez que queremos procesar los valores de sus componentes, tenemos que obtenerlos con los métodos provistos por el objeto. Por ejemplo, para obtener el año de una fecha, tenemos que usar el método `getFullYear()`. (ver **Listado 6-124**).

```
var hoy = new Date();  
var ano = hoy.getFullYear();  
alert("El año es " + ano); // "El año es 2017"
```

- El resto de los componentes se obtiene de la misma manera con los métodos respectivos. Lo único que tenemos que considerar es que los meses se representan con valores de 0 a 11, y, por lo tanto, tenemos que sumar 1 al valor que devuelve el método `getMonth()` para obtener un valor que las personas puedan entender (ver **Listado 6-125**).

```
var hoy = new Date();  
var ano = hoy.getFullYear();  
var mes = hoy.getMonth() + 1;  
var dia = hoy.getDate();  
alert(ano + "-" + mes + "-" + dia); // "2017-1-5"
```

# Objetos Date

- Estos métodos también resultan útiles cuando necesitamos incrementar o disminuir una fecha. Por ejemplo, si queremos determinar la fecha 15 días a partir de la fecha actual, tenemos que obtener el día actual, sumarle 15 y asignar el resultado al objeto.(ver Listado 6-126).

```
var hoy = new Date();  
alert(hoy); // "Thu Jan 05 2017 14:37:28 GMT-0500 (EST)"  
hoy.setDate(hoy.getDate() + 15);  
alert(hoy); // "Fri Jan 20 2017 14:37:28 GMT-0500 (EST)"
```

- Si en lugar de incrementar o disminuir una fecha por un período de tiempo, lo que queremos es calcular la diferencia entre dos fechas, tenemos que restar una fecha a la otra.(ver Listado 6-127)

```
var hoy = new Date(2017, 0, 5);  
var futuro = new Date(2017, 0, 20);  
var intervalo = futuro - hoy;  
alert(intervalo); // 1296000000
```

# Objetos Date

- El valor que devuelve la resta se expresa en milisegundos. A partir de este valor, podemos extraer cualquier componente que necesitemos. Todo lo que tenemos que hacer es dividir el número por los valores máximos de cada componente. Por ejemplo, si queremos expresar en número en segundos, tenemos que dividirlo por 1000 (1000 milisegundos = 1 segundo). (ver Listado 6-128).

```
var hoy = new Date(2017, 0, 5);  
var futuro = new Date(2017, 0, 20);  
var intervalo = futuro - hoy;  
var segundos = intervalo / 1000;  
alert(segundos + " segundos"); // "1296000 segundos"
```

# Objetos Date

- Por supuesto, no es fácil incluir un valor en segundos dentro de un contexto que podamos entender, pero sí podemos hacerlo con un valor en días. Para expresar el intervalo en días, tenemos que seguir dividiendo el resultado. Si dividimos los milisegundos por 1000, obtenemos segundos, el resultado dividido por 60 nos da los minutos, ese resultado dividido por 60 nuevamente nos da las horas, y dividiendo el resultado por 24 obtenemos los días (24 horas = 1 día). (ver Listado 6-129).

```
var hoy = new Date(2017, 0, 5);  
var futuro = new Date(2017, 0, 20);  
var intervalo = futuro - hoy;  
var 299áxi = intervalo / (24 * 60 * 60 * 1000);  
alert(299áxi + " días"); // "15 días"
```

# Objetos Date

- La comparación es otra operación útil que podemos realizar con fechas. Aunque podemos comparar objetos Date directamente, algunos operadores de comparación no comparan las fechas, sino los objetos mismos, por lo que la mejor manera de hacerlo es obtener primero las fechas en milisegundos con el método `getTime()` y luego comparar esos valores. (ver Listado 6-130).

```
var hoy = new Date(2017, 0, 20, 10, 35);
var futuro = new Date(2017, 0, 20, 12, 35);

if (futuro.getTime() == hoy.getTime()) {
    alert("Las Fechas son Iguales");
} else {
    alert("Las Fechas son Diferentes");
}
```

# Objetos Date

- Si queremos comparar solo la fecha, sin considerar la hora, tenemos que anular los componentes de la hora. Por ejemplo, podemos configurar las hora, los minutos y segundos en ambas fechas con el valor 0 y entonces los únicos valores a comparar serán el año, el mes y el día. (ver Listado 6-131).

```
var hoy = new Date(2017, 0, 20, 10, 35);
var futuro = new Date(2017, 0, 20, 12, 35);
hoy.setHours(0, 0, 0);
futuro.setHours(0, 0, 0);

if (futuro.getTime() == hoy.getTime()) {
    alert("Las Fechas son Iguales");
} else {
    alert("Las Fechas son Diferentes");
}
```

# Objetos Date

- Si queremos comparar solo la fecha, sin considerar la hora, tenemos que anular los componentes de la hora. Por ejemplo, podemos configurar las hora, los minutos y segundos en ambas fechas con el valor 0 y entonces los únicos valores a comparar serán el año, el mes y el día. (ver Listado 6-131).

```
var hoy = new Date(2017, 0, 20, 10, 35);
var futuro = new Date(2017, 0, 20, 12, 35);
hoy.setHours(0, 0, 0);
futuro.setHours(0, 0, 0);

if (futuro.getTime() == hoy.getTime()) {
    alert("Las Fechas son Iguales");
} else {
    alert("Las Fechas son Diferentes");
}
```

- En el Listado 6-131, antes de comparar las fechas, anulamos la hora, los minutos y los segundos con el método `setHours()`. Ahora, las fechas tienen la hora configurada en 0 y el operador de comparación solo tiene que comprobar si las fechas son iguales o no. En este caso, las fechas son iguales y el mensaje “Las fechas son Iguales” se muestra en pantalla.

# Objeto Math

Algunos objetos en JavaScript no tienen la función de almacenar valores, sino la de proveer propiedades y métodos para procesar valores de otros objetos. Este es el caso del objeto Math. Este objeto no incluye un constructor para crear más objetos, pero define varias propiedades y métodos a los que podemos acceder desde su definición para obtener los valores de constantes matemáticas y realizar operaciones aritméticas. Los siguientes son los más usados.

- **PI**—Esta propiedad devuelve el valor de PI.
- **E**—Esta propiedad devuelve la constante Euler.
- **LN10**—Esta propiedad devuelve el logaritmo natural de 10. El objeto también incluye las propiedades LN2 (algoritmo natural de 2), LOG2E (algoritmo base 2 de E) y LOG10E (algoritmo base 10 de E).
- **SQRT2**—Esta propiedad devuelve la raíz cuadrada de 2. El objeto también incluye la propiedad SQRT1\_2 (la raíz cuadrada de 1/2).
- **Ceil(valor)**—Este método redondea un valor hacia arriba al siguiente entero y devuelve el resultado.
- **Floor(valor)**—Este método redondea un valor hacia abajo al siguiente entero y devuelve el resultado.

# Objeto Math

- **Round(valor)**—Este método redondea un valor al entero más cercano y devuelve el resultado.
- **Trunc(valor)**—Este método elimina los dígitos de la fracción y devuelve un entero.
- **abs(valor)**—Este método devuelve el valor absoluto de un número (invierte valores negativos para obtener un número positivo).
- **Min(valor)**—Este método devuelve el número más pequeño de una lista de valores separados por comas.
- **Max(valores)**—Este método devuelve el número más grande de una lista de valores separados por comas.
- **Random()**—Este método devuelve un número al azar en un rango entre 0 y 1.
- **Pow(base, exponente)**—Este método devuelve el resultado de elevar la base a la potencia del exponente.
- **Exp(exponente)**—Este método devuelve el resultado de elevar E a la potencia del exponente. El objeto también incluye el método `expm1()`, que devuelve el mismo resultado menos 1.
- **Sqrt(valor)**—Este método devuelve la raíz cuadrada de un valor.

# Objeto Math

- **Log10(valor)**—Este método devuelve el logaritmo base 10 de un valor. El objeto también incluye los métodos `log()` (devuelve el logaritmo base E), `log2()` (devuelve el logaritmo base 2) y `log1p()` (devuelve el logaritmo base E de 1 más un número).
- **Sin(valor)**—Este método devuelve el seno de un número. El objeto también incluye los métodos `asin()` (devuelve el arcoseno de un número), `sinh()` (devuelve el seno hiperbólico de un número) y `asinh()` (devuelve el arcoseno hiperbólico de un número).
- **Cos(valor)**—Este método devuelve el coseno de un número. El objeto también incluye los métodos `acos()` (devuelve el arcocoseno de un número), `cosh()` (devuelve el coseno hiperbólico de un número), y `acosh()` (devuelve el arcocoseno hiperbólico de un número).
- **Tan(valor)**—Este método devuelve la tangente de un número. El objeto también incluye los métodos `atan()` (devuelve la arcotangente de un número), `atan2()` (devuelve la arcotangente o el cociente de dos números), `tanh()` (devuelve la tangente hiperbólica de un número), y `atanh()` (devuelve la arcotangente hiperbólica de un número).

# Objeto Math

- Como no se crea ninguna instancia de este objeto, debemos leer las propiedades y llamar a los métodos desde el objeto mismo (por ejemplo, Math.PI). Aparte de esto, la aplicación de estos métodos y valores es sencilla, tal como muestra el siguiente ejemplo (ver Listado 6-132).

```
var cuadrado = Math.sqrt(4); // 2
var elevado = Math.pow(2, 2); // 4
var 301áximo = Math.max(cuadrado, elevado);

alert("El valor más grande es " + 301áximo); // "El valor más grande
es 4"
```

- El siguiente ejemplo demuestra cómo calcular un número al azar (ver Listado 6-133).

```
var numeroalazar = Math.random() * (11 - 1) + 1;
var valor = Math.floor(numeroalazar);
alert("El número es: " + valor);
```

El método `random()` devuelve un número entre 0 y 1. Si queremos obtener un número dentro de un rango diferente de valores, tenemos que multiplicar el valor por la fórmula  $(\text{max} - \text{min}) + \text{min}$ , donde min y max son los valores mínimos y máximos que queremos incluir. En nuestro ejemplo, queremos obtener un número al azar entre 1 y 10, pero hemos tenido que definirlo como 1 y 11 debido a que el valor máximo no se incluye en el rango. Otro tema que debemos considerar es que el número que devuelve el método `random()` es un valor decimal. Si queremos obtener un número entero, tenemos que redondearlo con el método `floor()`.

# Objeto Window

- Cada vez que abrimos el navegador o iniciamos una nueva pestaña, se crea un objeto global llamado Window para referenciar la ventana del navegador y proveer algunas propiedades y métodos esenciales. El objeto se almacena en una propiedad del objeto global de JavaScript llamada window. A través de esta propiedad podemos conectarnos con el navegador y el documento desde nuestro código.
- El objeto Window a su vez incluye otros objetos con los que provee información adicional relacionada con la ventana y el documento. Las siguientes son algunas de las propiedades disponibles para acceder a estos objetos.

# Objeto Window

- **Location**—Esta propiedad contiene un objeto Location con información acerca del origen del documento. También se puede usar como una propiedad para declarar o devolver la URL del documento (por ejemplo, `window.location = "http://www.formasterminds.com"`).
- **history**—Esta propiedad contiene un objeto History con propiedades y métodos para manipular el historial de navegación.
- **Navigator**—Esta propiedad contiene un objeto Navigator con información acerca de la aplicación y el dispositivo. Estudiaremos algunas de las propiedades de este objeto, como geolocation (usada para detectar la ubicación del usuario).
- **Document**—Esta propiedad contiene un objeto Document, que provee acceso a los objetos que representan los elementos HTML en el documento.

# Objeto Window

Además de estos valiosos objetos, el objeto Window también ofrece sus propias propiedades y métodos. Los siguientes son los que más se usan:

- **innerWidth**—Esta propiedad devuelve el ancho de la ventana en píxeles.
- **innerHeight**—Esta propiedad devuelve la altura de la ventana en píxeles.
- **scrollX**—Esta propiedad devuelve el número de píxeles en los que el documento se ha desplazado horizontalmente.
- **scrollY**—Esta propiedad devuelve el número de píxeles en los que el documento se ha desplazado verticalmente.
- **Alert(valor)**—Este método muestra una ventana emergente en la pantalla que muestra el valor entre paréntesis.
- **Confirm(mensaje)**—Este método es similar a `alert()`, pero ofrece dos botones, Aceptar y Cancelar, para que el usuario elija qué hacer. El método devuelve `true` o `false`, según a la respuesta del usuario.

# Objeto Window

- **Prompt(mensaje)**—Este método muestra una ventana emergente con un campo de entrada para permitir al usuario introducir un valor. El método devuelve el valor que inserta el usuario.
- **setTimeout(función, milisegundos)**—Este método ejecuta la función especificada en el primer atributo cuando haya pasado el tiempo especificado por el segundo atributo. El objeto Window también ofrece el método `clearTimeout()` para cancelar este proceso.
- **setInterval(función, milisegundos)**—Este método es similar a `setTimeout()`, pero llama a la función constantemente. El objeto Window también ofrece el método `clearInterval()` para cancelar el proceso.
- **Open(URL, ventana, parámetros)**—Este método abre un documento en una nueva ventana. El atributo **URL** es la URL del documento que queremos abrir, el atributo **ventana** es el nombre de la ventana donde queremos mostrar el documento (si el nombre no se especifica o la ventana no existe, el documento se abre en una nueva ventana), y el atributo **parámetros** es una lista de parámetros de configuración separados por comas que configuran las características de la ventana (por ejemplo, “resizable=no,scrollbars=no”). El objeto Window también ofrece el método `close()` para cerrar una ventana abierta con este método.

# Objeto Window

- El objeto Window controla aspectos de la ventana, su contenido, y los datos asociados a la misma, como la ubicación del documento actual, el tamaño, el desplazamiento, etc. Por ejemplo, podemos cargar un nuevo documento cambiando el valor de la ubicación (ver listado6-134).

```
1  <!DOCTYPE html>
2  <html lang="es">
3      <head>
4          <meta charset="utf-8">
5          <title>JavaScript</title>
6          <script>
7              function realizar() {
8                  window.location = "http://www.formasterminds.com";
9              }
10         </script>
11     </head>
12     <body>
13         <section>
14             <h1>Sitio Web</h1>
15             <button type="button" onclick="realizar()">Presione Aquí</button>
16         </section>
17     </body>
18 </html>
```

# Objeto Window

Además de asignar una nueva URL a la propiedad location, también podemos manipular la ubicación desde los métodos provistos por el objeto Location.

- **Assign(URL)**—Este método le pide al navegador que cargue el documento en la ubicación especificada por el atributo **URL**.
- **Replace(URL)**—Este método le pide al navegador que reemplace el documento actual con el documento en la ubicación indicada por el atributo **URL**. Difiere del método assign() en que no agrega la URL al historial del navegador.
- **Reload(valor)**—Este método le pide al navegador que actualice el documento actual. Acepta un valor booleano que determina si el recurso se tiene que descargar desde el servidor o se puede cargar desde el caché del navegador (true o false).

# Objeto Window

- El siguiente ejemplo actualiza la página cuando el usuario pulsa un botón. Esta vez no mencionamos la propiedad window. El objeto Window es un objeto global y, por lo tanto, el intérprete infiere que las propiedades y los métodos pertenecen a este objeto. Esta es la razón por la que en anteriores ejemplos nunca hemos llamado al método `alert()` con la instrucción `window.alert()` (ver listado6-135).

```
1  <!DOCTYPE html>
2  <html lang="es">
3      <head>
4          <meta charset="utf-8">
5          <title>JavaScript</title>
6          <script>
7              function realizar() {
8                  location.reload();
9              }
10         </script>
11     </head>
12     <body>
13         <section>
14             <h1>Sitio Web</h1>
15             <button type="button" onclick="realizar()">Presione Aquí</button>
16         </section>
17     </body>
18 </html>
```

# Objeto Window

- El objeto Window también ofrece el método `open()` para cargar nuevo contenido. En el siguiente ejemplo, el sitio web `www.formasterminds.com` se abre en una nueva ventana o pestaña. (ver listado 6-136).

```
1  <!DOCTYPE html>
2  <html lang="es">
3      <head>
4          <meta charset="utf-8">
5          <title>JavaScript</title>
6          <script>
7              function realizar() {
8                  open("http://www.formasterminds.com");
9              }
10         </script>
11     </head>
12     <body>
13         <section>
14             <h1>Sitio Web</h1>
15             <button type="button" onclick="realizar()">Presione Aquí</button>
16         </section>
17     </body>
18 </html>
```

# Objeto Window

Otros métodos del objeto Window son `setTimeout()` y `setInterval()`. Estos métodos ejecutan una instrucción después de un cierto período de tiempo. El método `setTimeout()` ejecuta la instrucción una vez, y el método `setInterval()` ejecuta la instrucción de forma repetida hasta que el proceso se cancela. Si en lugar de una instrucción queremos ejecutar varias, podemos especificar una referencia a una función. Cada vez que el tiempo finaliza, la función se ejecuta. (ver listado6-137).

```
1  <!DOCTYPE html>
2  <html lang="es">
3      <head>
4          <meta charset="utf-8">
5          <title>JavaScript</title>
6          <script>
7              function realizar() {
8                  var hoy = new Date();
9                  var tiempo = hoy.toString();
10                 alert(tiempo);
11             }
12         </script>
13     </head>
14     <body>
15         <section>
16             <h1>Sitio Web</h1>
17             <button type="button" onclick="setTimeout(realizar, 5000)">Presione Aquí</button>
18         </section>
19     </body>
20 </html>
```

# Objeto Window

Si lo que necesitamos es ejecutar la función una y otra vez después de un período de tiempo, tenemos que usar el método `setInterval()`. Este método trabaja exactamente como `setTimeout()` pero sigue funcionando hasta que le pedimos que se detenga con el método `clearInterval()`. Para identificar al método que queremos que se cancele, tenemos que almacenar la referencia que devuelve el método `setInterval()` en una variable y usar esa variable para referenciar el método más adelante, tal como hacemos en el siguiente ejemplo.(ver listado6-138).

```
<!DOCTYPE html>
<html lang="es">
  <head>
    <meta charset="utf-8">
    <title>JavaScript</title>
    <script>
      var segundos = 0;
      var contador = setInterval(realizar, 1000);
      function realizar() {
        segundos++;
      }
      function cancelar() {
        clearInterval(contador);
        alert("Total: " + segundos + " segundos");
      }
    </script>
  </head>
  <body>
    <section>
      <h1>Sitio Web</h1>
      <button type="button" onclick="cancelar()">Presione Aquí</button>
    </section>
  </body>
</html>
```

# Objeto Document

- Como ya hemos mencionado, casi todo en JavaScript se define como un objeto, y esto incluye los elementos en el documento. Cuando se carga un documento HTML, el navegador crea una estructura interna para procesarlo. La estructura se llama *DOM* (Document Object Model) y está compuesta por múltiples objetos de tipo Element (u otros tipos más específicos que heredan de Element), que representan cada elemento en el documento.
- Los objetos Element mantienen una conexión permanente con los elementos que representan. Cuando se modifica un objeto, su elemento también se modifica y el resultado se muestra en pantalla. Para ofrecer acceso a estos objetos y permitirnos alterar sus propiedades desde nuestro código JavaScript, los objetos se almacenan en un objeto llamado Document que se asigna a la propiedad document del objeto Window.
- Entre otras alternativas, el objeto Document incluye las siguientes propiedades para ofrecer acceso rápido a los objetos Element que representan los elementos más comunes del documento.

# Objeto Document

- forms—Esta propiedad devuelve un array con referencias a todos los objetos Element que representan los elementos <form> en el documento.
- images—Esta propiedad devuelve un array con referencias a todos los objetos Element que representan los elementos <img> en el documento.
- links—Esta propiedad devuelve un array con referencias a todos los objetos Element que representan los elementos <a> en el documento.

Estas propiedades devuelven un arreglo de objetos que referencian todos los elementos de un tipo particular.

# Objeto Document

Los siguientes métodos acceder a objetos individuales u obtener listas de objetos a partir de otros parámetros.

- **getElementById(id)** — este método devuelve una referencia al objeto **Element** que representa el elemento identificado con el valor especificado por el atributo.
- **getElementsByClassName(clase)** — Este método devuelve un array con referencias a los objetos **Element** que representan los elementos identificados con la clase especificada por el atributo (el valor asignado al atributo **clase**).
- **getElementsByName(nombre)** — Este método devuelve un array con referencias a los objetos **Element** que representan los elementos identificados con el nombre especificado por el atributo (el valor asignado al atributo **name**).
- **getElementsByTagName(tipo)** — Este método devuelve un array con referencias a los objetos **Element** que representan el tipo de elementos especificados por el atributo. El atributo es el nombre que identifica a cada tipo de elemento, como **h1**, **p**, **img**, **div**, etc
- **querySelector(selector)** — Este método devuelve una referencia al objeto **Element** que representa el elemento que coincide con el selector especificado por el atributo. El método devuelve el primer elemento en el documento que coincide con el selector CSS
- **querySelectorAll(selectores)** — Este método devuelve un array con referencias a los objetos **Element** que representan los elementos que coinciden con los selectores especificados por el atributo. Se pueden declarar uno o más selectores separados por comas.

Accediendo a los objetos **Element** en el DOM, podemos leer y modificar los elementos en el documento, pero antes de hacerlo, debemos considerar que el navegador lee el documento de forma secuencial y no podemos referenciar un elemento que aún no se ha creado. La mejor solución a este problema es ejecutar el código JavaScript solo cuando ocurre el evento **load**.

```
1  <!DOCTYPE html>
2  <html lang="es">
3      <head>
4          <meta charset="utf-8">
5          <title>Objeto Document</title>
6          <script>
7              function iniciar(){
8                  var elemento = document.getElementById("subtitulo");
9              }
10         </script>
11     </head>
12     <body onload="iniciar()">
13         <section>
14             <h1>WebSite</h1>
15             <p id="subtitulo">el mejor sitio web!</p>
16         </section>
17     </body>
18 </html>
```

```
1  <!DOCTYPE html>
2  <html lang="es">
3      <head>
4          <meta charset="utf-8">
5          <title>Objeto Document</title>
6          <script>
7              function iniciar(){
8                  var elemento = document.getElementById("subtitulo");
9                  alert("el id del elemento es: " + elemento.id);
10             }
11         </script>
12     </head>
13     <body onload="iniciar()">
14         <section>
15             <h1>WebSite</h1>
16             <p id="subtitulo">el mejor sitio web!</p>
17         </section>
18     </body>
```

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="utf-8">
5      <title>JavaScript</title>
6      <script>
7          function iniciar() {
8              var lista = document.getElementsByTagName("p");
9              for (var f = 0; f < lista.length; f++) {
10                  var elemento = lista[f];
11                  alert("El id es: " + elemento.id);
12              }
13          }
14      </script>
15  </head>
16  <body onload="iniciar()">
17      <section>
18          <h1>Sitio Web</h1>
19          <p id="subtitulo">El mejor sitio web!</p>
20          <p id="subtitulo1">El mejor sitio web1!</p>
21          <p id="subtitulo2">El mejor sitio web2!</p>
22      </section>
23  </body>
24  </html>
```

---

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="utf-8">
5      <title>JavaScript</title>
6      <script>
7          function iniciar() {
8              var lista = document.getElementsByTagName("p");
9              var elemento = lista[0];
10             alert("El id es: " + elemento.id);
11         }
12     </script>
13 </head>
14 <body onload="iniciar()">
15     <section>
16         <h1>Sitio Web</h1>
17         <p id="subtitulo">El mejor sitio web!</p>
18     </section>
19 </body>
20 </html>
```

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="utf-8">
5      <title>JavaScript</title>
6      <script>
7          function iniciar() {
8              var elemento = document.querySelector("section > p");
9              alert("El id es: " + elemento.id);
10         }
11     </script>
12 </head>
13 <body onload="iniciar()">
14     <section>
15         <h1>Sitio Web</h1>
16         <p id="subtitulo">El mejor sitio web!</p>
17     </section>
18 </body>
19 </html>
```

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="utf-8">
5      <title>JavaScript</title>
6      <script>
7          function iniciar() {
8              var elemprincipal = document.getElementById("seccionprincipal");
9              var lista = elemprincipal.getElementsByTagName("p");
10             var elemento = lista[0];
11             alert("El id es: " + elemento.id);
12         }
13     </script>
14 </head>
15 <body onload="iniciar()">
16     <section id="seccionprincipal">
17         <h1>Sitio Web</h1>
18         <p id="subtitulo">El mejor sitio web!</p>
19     </section>
20 </body>
21 </html>
```

# Objetos Element

- Obtener una referencia para acceder a un elemento y leer sus atributos puede ser útil en algunas circunstancias, pero lo que convierte a JavaScript en un lenguaje dinámico es la posibilidad de modificar esos elementos y el documento. Con este propósito, los objetos **Element** contienen propiedades para manipular y definir los estilos de los elementos y contenidos. Una de estas propiedades es **style**, el cual contiene un objeto llamado **Styles** que a su vez incluye propiedades para modificar los estilos de los elementos.
- Las siguientes son las propiedades que más se usan.

- **color** – Esta propiedad declara el color del contenido del elemento.
- **background** – Esta propiedad declara los estilos del fondo del elemento. También podemos trabajar con cada estilo de forma independiente usando las propiedades asociadas **backgroundColor**, **backgroundImage**, **backgroundRepeat**, **backgroundPosition** y **backgroundAttachment**.
- **border** – Esta propiedad declara los estilos del borde del elemento. Podemos modificar cada estilo de forma independiente con las propiedades asociadas **borderColor**, **borderStyle**, y **borderWidth**, o modificar cada borde individualmente usando las propiedades asociadas **borderTop**, **borderBottom**, **borderLeft**.
- **margin** – Esta propiedad declara el margen del elemento. También podemos usar las propiedades asociadas **marginBottom**, **marginLeft**, **marginRight**, y **marginTop**.
- **padding** - propiedad declara el relleno del elemento. También podemos usar las propiedades asociadas **paddingBottom**, **paddingLeft**, **paddingRight**, y **paddingTop**.

- **width** – Esta propiedad declara el ancho del elemento. Existen dos propiedades asociadas para declarar el ancho máximo y mínimo de un elemento: **maxWidth** y **minWidth**.
- **height** - Esta propiedad declara la altura del elemento. Existen dos propiedades asociadas **maxHeight** y **minHeight**.
- **visibility** - Esta propiedad determina si el elemento es visible o no
- **display** - Esta propiedad define el tipo de caja usado para presentar el elemento.
- **position** - Esta propiedad define el tipo de posicionamiento usado para posicionar elemento.
- **top** – Esta propiedad especifica la distancia entre el margen superior del elemento y el margen superior de su contenedor.
- **bottom** - Esta propiedad especifica la distancia entre el margen inferior del elemento y el margen inferior de su contenedor.
- **left** – Esta propiedad especifica la distancia entre el margen izquierdo del elemento y el margen izquierdo de su contenedor.

**right**—Esta propiedad especifica la distancia entre el margen derecho del elemento y el margen derecho de su contenedor.

**cssFloat**—Esta propiedad permite al elemento flotar hacia un lado o el otro.

**clear**—Esta propiedad recupera el flujo normal del documento, impidiendo que el elemento siga flotando hacia la izquierda, la derecha, o ambos lados.

**overflow**—Esta propiedad especifica cómo se va a mostrar el contenido que excede los límites de la caja de su contenedor.

**zIndex**—Esta propiedad define un índice que determina la posición del elemento en el eje z.

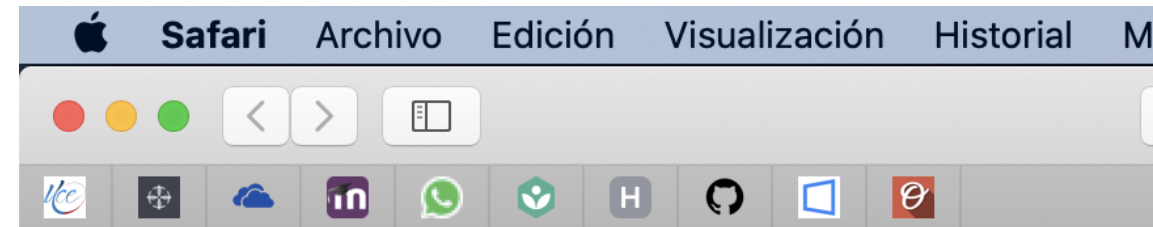
**font**—Esta propiedad declara los estilos de la fuente. También podemos declarar los estilos individuales usando las propiedades asociadas `fontFamily`, `fontSize`, `fontStyle`, `fontVariant` y `fontWeight`.

**textAlign**—Esta propiedad alinea el texto dentro del elemento.

**verticalAlign**—Esta propiedad alinea elementos Inline verticalmente.

**textDecoration**—Esta propiedad resalta el texto con una línea. También podemos declarar los estilos de forma individual asignando los valores `true` o `false` a las propiedades `textDecorationBlink`, `textDecorationLineThrough`, `textDecorationNone`, `textDecorationOverline`, y `textDecorationUnderline`.

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="utf-8">
5      <title>JavaScript</title>
6      <script>
7          function iniciar() {
8              var elemento = document.getElementById("subtitulo");
9              elemento.style.width = "300px";
10             elemento.style.border = "1px solid #FF0000";
11             elemento.style.padding = "20px";
12         }
13     </script>
14 </head>
15 <body onload="iniciar()">
16     <section>
17         <h1>Sitio Web</h1>
18         <p id="subtitulo">El mejor sitio web!</p>
19     </section>
20 </body>
21 </html>
```



## Sitio Web

El mejor sitio web!

Las propiedades del objeto Styles son independientes de los estilos CSS asignados al documento. Si intentamos leer alguna de estas propiedades pero aún no le hemos asignado ningún valor desde JavaScript, el valor que devuelve será una cadena de caracteres vacía. Para proveer información acerca del elemento, los objetos Element incluyan las siguientes propiedades:

- `clientWidth` – devuelve al ancho del elemento, incluido el relleno.
- `clientHeight` – devuelve la altura del elemento, incluido el relleno.
- `offsetTop` – devuelve el número de píxeles que se ha desplazado el elemento desde la parte superior de su contenedor.
- `offsetLeft` – devuelve el número de píxeles que se ha desplazado el elemento desde el lado izquierdo de su contenedor.

- `offsetWidth` – devuelve el ancho del elemento, incluidos el relleno y el borde.
- `offsetHeight` – devuelve la altura del elemento, incluidos el relleno y el borde.
- `scrollTop` – devuelve el número de píxeles en los que el contenido del elemento se ha desplazado hacia arriba.
- `scrollLeft` – devuelve el número de píxeles en los que el contenido del elemento se ha desplazado hacia la izquierda.
- `scrollWidth` – devuelve el ancho del contenido del elemento.
- `scrollHeight` – devuelve la altura del contenido del elemento.

Estas propiedades son solo de lectura, podemos obtener el valor leyendo estas propiedades y después usar las propiedades del objeto `Styles` para asignarle uno nuevo.

```

1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="utf-8">
5      <title>JavaScript</title>
6      <style>
7          #subtitulo {
8              width: 300px;
9              padding: 20px;
10             border: 1px solid ■#FF0000;
11         }
12     </style>
13     <script>
14         function iniciar() {
15             var elemento = document.getElementById("subtitulo");
16             var ancho = elemento.clientWidth;
17             ancho = ancho + 100;
18             elemento.style.width = ancho + "px";
19         }
20     </script>
21 </head>
22 <body onload="iniciar()">
23     <section>
24         <h1>Sitio Web</h1>
25         <p id="subtitulo">El mejor sitio web!</p>
26     </section>
27 </body>
28 </html>

```



## Sitio Web

El mejor sitio web!

Reemplazar una clase.

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="utf-8">
5      <title>JavaScript</title>
6      <style>
7          .supercolor {
8              background: ■ #0099EE;
9          }
10         .colornegro {
11             background: ■ #000000;
12         }
13     </style>
14     <script>
15         function cambiarcolor() {
16             var elemento = document.getElementById("subtitulo");
17             elemento.className = "colornegro";
18         }
19     </script>
20 </head>
21 <body>
22     <section>
23         <h1>Sitio Web</h1>
24         <p id="subtitulo" class="supercolor" onclick="cambiarcolor()">El mejor sitio web!</p>
25     </section>
26 </body>
27 </html>
```

Asignar varias clases a un mismo elemento.

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="utf-8">
5      <title>JavaScript</title>
6      <style>
7          .supercolor {
8              background: ■ #000000;
9          }
10     </style>
11     <script>
12         function cambiarcolor() {
13             var elemento = document.getElementById("subtitulo");
14             if (elemento.classList.contains("supercolor")) {
15                 elemento.classList.remove("supercolor");
16             } else {
17                 elemento.classList.add("supercolor");
18             }
19         }
20     </script>
21 </head>
22 <body>
23     <section>
24         <h1>Sitio Web</h1>
25         <p id="subtitulo" class="supercolor" onclick="cambiarcolor()">El mejor sitio web!</p>
26     </section>
27 </body>
28 </html>
```

Se puede usar el método `toggle()` para asignar varias clases a un mismo elemento. Con este método ya no tenemos que preocuparnos si la clase existe o no, ya que si no existe la crea o la elimina.

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="utf-8">
5      <title>JavaScript</title>
6      <style>
7          .supercolor {
8              background: ■ #000000;
9          }
10     </style>
11     <script>
12         function cambiarcolor() {
13             var elemento = document.getElementById("subtitulo");
14             elemento.classList.toggle("supercolor");
15         }
16     </script>
17 </head>
18 <body>
19     <section>
20         <h1>Sitio Web</h1>
21         <p id="subtitulo" class="supercolor" onclick="cambiarcolor()">El mejor sitio web!</p>
22     </section>
23 </body>
24 </html>
```

- Además de los estilos de un elemento, también podemos modificar su contenido. Algunas propiedades y métodos para este propósito:
- `innerHTML` – esta propiedad declara o devuelve el contenido de un elemento.
- `outerHTML` – esta propiedad declara o devuelve un elemento y su contenido, esta propiedad reemplaza no solo el contenido, sino también el elemento.
- `insertAdjacentHTML(ubicación, contenido)` – inserta contenido en una ubicación determinada por el atributo ubicación, los valores disponibles son `beforebegin` (antes del elemento), `afterbegin` (dentro del elemento, antes del primer elemento hijo), `beforeend` (dentro del elemento, después del último elemento hijo) y `afterend` (después deñ elemento).

innerHTML, permite  
reemplazar el contenido de  
un elemento.

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="utf-8">
5      <title>JavaScript</title>
6      <script>
7          function cambiarcontenido() {
8              var elemento = document.getElementById("subtitulo");
9              elemento.innerHTML = "Este es mi sitio web";
10         }
11     </script>
12 </head>
13 <body>
14     <section>
15         <h1>Sitio Web</h1>
16         <p id="subtitulo" onclick="cambiarcontenido()">El mejor sitio web!</p>
17     </section>
18 </body>
19 </html>
```

innerHTML, permite leer y procesar el contenido actual.

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="utf-8">
5      <title>JavaScript</title>
6      <script>
7          function cambiarcontenido() {
8              var elemento = document.getElementById("subtitulo");
9              var texto = elemento.innerHTML + " Somos los mejores!";
10             elemento.innerHTML = texto;
11         }
12     </script>
13 </head>
14 <body>
15     <section>
16         <h1>Sitio Web</h1>
17         <p id="subtitulo" onclick="cambiarcontenido()">El mejor sitio web!</p>
18     </section>
19 </body>
20 </html>
```

innerHTML, también puede interpretar el código HTML

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="utf-8">
5      <title>JavaScript</title>
6      <script>
7          function agregarelemento() {
8              var elemento = document.querySelector("section");
9              elemento.innerHTML = "<p>Este es un texto nuevo</p>";
10         }
11     </script>
12 </head>
13 <body>
14     <section>
15         <h1>Sitio Web</h1>
16         <button type="button" onclick="agregarelemento()">Agregar Contenido</button>
17     </section>
18 </body>
19 </html>
```

insertAdjacentHTML(), no reemplaza el contenido, sino que permite agregar más contenido antes o después del contenido actual y también fuera del documento, esto quiere decir que agrega contenido al documento, pero sin que afecte al contenido existente.

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="utf-8">
5      <title>JavaScript</title>
6      <script>
7          function agregarelemento() {
8              var elemento = document.querySelector("section");
9              elemento.insertAdjacentHTML("beforeend", "<p>Este es un texto nuevo</p>");
10         }
11     </script>
12 </head>
13 <body>
14     <section>
15         <h1>Sitio Web</h1>
16         <button type="button" onclick="agregarelemento()">Agregar Contenido</button>
17     </section>
18 </body>
19 </html>
```

# Creando objetos Element

El objeto Document incluye métodos para trabajar directamente con los objetos Element.

- `createElement(nombre)` – crea un nuevo objeto Element del tipo especificado por el atributo nombre.
- `appendChild(elemento)` – inserta el elemento representado por un objeto Element como hijo de un elemento existente en el documento.
- `removeChild(elemento)` – elimina un elemento hijo de otro elemento.

Para crear un nuevo objeto Element y poder usarlo, primero lo creamos con el método createElement() y luego agregamos el elemento al documento con el método appendChild().

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="utf-8">
5      <title>JavaScript</title>
6      <script>
7          function agregarelemento() {
8              var elemento = document.querySelector("section");
9              var elementonuevo = document.createElement("p");
10             elemento.appendChild(elementonuevo);
11         }
12     </script>
13 </head>
14 <body>
15     <section>
16         <h1>Sitio Web</h1>
17         <button type="button" onclick="agregarelemento()">Agregar Elemento</button>
18     </section>
19 </body>
20 </html>
```

Asignamos un contenido al objeto Element, antes de agregar el elemento al documento.

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="utf-8">
5      <title>JavaScript</title>
6      <script>
7          function agregarelemento() {
8              var elemento = document.querySelector("section");
9              var elementonuevo = document.createElement("p");
10             elementonuevo.innerHTML = "Este es un elemento nuevo";
11             elemento.appendChild(elementonuevo);
12         }
13     </script>
14 </head>
15 <body>
16     <section>
17         <h1>Sitio Web</h1>
18         <button type="button" onclick="agregarelemento()">Agregar Elemento</button>
19     </section>
20 </body>
21 </html>
```

# Eventos

Desde JavaScript podemos definir atributos de eventos sobre los elementos del documento. Por ejemplo, si queremos responder al evento click, solo tenemos que definir la propiedad onclick del elemento.

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="utf-8">
5      <title>JavaScript</title>
6      <script>
7          function agregarevento() {
8              var elemento = document.querySelector("section > button");
9              elemento.onclick = mostrarmensaje;
10         }
11         function mostrarmensaje() {
12             alert("Presionó el botón");
13         }
14         window.onload = agregarevento;
15     </script>
16 </head>
17 <body>
18     <section>
19         <h1>Sitio Web</h1>
20         <button type="button">Mostrar</button>
21     </section>
22 </body>
23 </html>
```

# Le método `addEventListener()`

Hay nuevos métodos en el objeto `Window` para controlar y responder a eventos.

- `addEventListener(evento, listener, captura)` – prepara a un elemento para responder a un evento. El atributo `evento` es el nombre del evento, `listener` es una referencia a la función que responderá al evento y `captura` es un booleano que determina si el evento va a ser capturado por el elemento o se propagará a otros elementos.
- `removeEventListener(evento, listener)` – elimina el listener de un elemento.

El siguiente ejemplo hace lo mismo que el anterior, pero utilizando el método `addEventListener()` para agregar listeners al objeto `Window` y el elemento `<button>`

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="utf-8">
5      <title>JavaScript</title>
6      <script>
7          function agregarevento() {
8              var elemento = document.querySelector("section > button");
9              elemento.addEventListener("click", mostrarmensaje);
10         }
11         function mostrarmensaje() {
12             alert("Presionó el botón");
13         }
14         window.addEventListener("load", agregarevento);
15     </script>
16 </head>
17 <body>
18     <section>
19         <h1>Sitio Web</h1>
20         <button type="button">Mostrar</button>
21     </section>
22 </body>
23 </html>
```

# Objetos Event

- Cada función que responde a un evento recibe un objeto que contiene información acerca del evento. Aunque algunos eventos tienen sus propios objetos, existe un objeto llamado Event que es común a cada evento. Tiene las siguientes propiedades y métodos.
- target – devuelve una referencia al objeto que ha recibido el evento.
- type – devuelve una cadena de caracteres con el nombre del evento.
- preventDefault() – cancela el evento para prevenir que el sistema realice tareas por defecto.
- stopPropagation() – detiene la propagación del evento a otros elementos, de modo que solo el primer elemento que recibe el evento puede procesarlo.

Usamos el objeto Event para identificar el elemento en que el usuario a hecho clic.

Para identificar en que elemento ha hecho clic el usuario, leemos la propiedad target del objeto Event.

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="utf-8">
5      <title>JavaScript</title>
6      <script>
7          function agregareventos() {
8              var lista = document.querySelectorAll("section > p");
9              for (var f = 0; f < lista.length; f++) {
10                 var elemento = lista[f];
11                 elemento.addEventListener("click", cambiarcolor);
12             }
13         }
14         function cambiarcolor(evento) {
15             var elemento = evento.target;
16             elemento.style.backgroundColor = "#999999";
17         }
18         window.addEventListener("load", agregareventos);
19     </script>
20 </head>
21 <body>
22     <section>
23         <h1>Sitio Web</h1>
24         <p>Mensaje número 1</p>
25         <p>Mensaje número 2</p>
26         <p>Mensaje número 3</p>
27     </section>
28 </body>
29 </html>
```

Para enviar nuestros propios valores junto con el objeto Event que corresponda, se puede procesar el evento con una función anónima.

```
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4    <meta charset="utf-8">
5    <title>JavaScript</title>
6    <script>
7      function agregareventos() {
8        var lista = document.querySelectorAll("section > p");
9        for (var f = 0; f < lista.length; f++) {
10          var elemento = lista[f];
11          elemento.addEventListener("click", function(evento) {
12            var mivalor = 125;
13            cambiarcolor(evento, mivalor);
14          });
15        }
16      }
17      function cambiarcolor(evento, mivalor) {
18        var elemento = evento.target;
19        elemento.innerHTML = "Valor " + mivalor;
20      }
21      window.addEventListener("load", agregareventos);
22    </script>
23  </head>
24  <body>
25    <section>
26      <h1>Sitio Web</h1>
27      <p>Mensaje número 1</p>
28      <p>Mensaje número 2</p>
29      <p>Mensaje número 3</p>
30    </section>
31  </body>
32  </html>
```

Si intentamos pasare el valor de una variable, podemos trabajar con bucles para ejemplarizar, pero tendremos un problema, debido a que la instrucción no se procesa hasta que el usuario hace clic en el elemento, el sistema asigna una referencia de la variable f a mivalor, no su valor actual. Esto significa que el sistema va a leer la variable f y asignar su valor a la variable mivalor solo cuando el evento clic se desencadena, para entonces el buble for ya habrá finalizado, por lo que el valor de f será 3.

```
1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <meta charset="utf-8">
5   <title>JavaScript</title>
6   <script>
7     function agregareventos() {
8       var lista = document.querySelectorAll("section > p");
9       for (var f = 0; f < lista.length; f++) {
10         var elemento = lista[f];
11         elemento.addEventListener("click", function(evento) {
12           var mivalor = f;
13           cambiarcolor(evento, mivalor);
14         });
15       }
16     }
17     function cambiarcolor(evento, mivalor) {
18       var elemento = evento.target;
19       elemento.innerHTML = "Valor " + mivalor;
20     }
21     window.addEventListener("load", agregareventos);
22   </script>
23 </head>
24 <body>
25   <section>
26     <h1>Sitio Web</h1>
27     <p>Mensaje número 1</p>
28     <p>Mensaje número 2</p>
29     <p>Mensaje número 3</p>
30   </section>
31 </body>
32 </html>
```

Podmos resolver el problema anterior, con dos funciones anónimas combinadas. Una función se ejecuta de inmediato, y la otra es la que devuelve la primera. La función principal debe recibir el valor actual de f, almacenarlo en otra variable y luego devolver una segunda función anónima con estos valores, la función anónima devuelta es la que se ejecutará cuando ocurra el evento.

```
1 <!DOCTYPE html>
2 <html lang="es">
3 <head>
4   <meta charset="utf-8">
5   <title>JavaScript</title>
6   <script>
7     function agregarEventos() {
8       var lista = document.querySelectorAll("section > p");
9       for (var f = 0; f < lista.length; f++) {
10         var elemento = lista[f];
11         elemento.addEventListener("click", function(x) {
12           return function(evento) {
13             var mivalor = x;
14             cambiarColor(evento, mivalor);
15           };
16         })(f));
17       }
18     }
19     function cambiarColor(evento, mivalor) {
20       var elemento = evento.target;
21       elemento.innerHTML = "Valor " + mivalor;
22     }
23     window.addEventListener("load", agregarEventos);
24   </script>
25 </head>
26 <body>
27   <section>
28     <h1>Sitio Web</h1>
29     <p>Mensaje número 1</p>
30     <p>Mensaje número 2</p>
31     <p>Mensaje número 3</p>
32   </section>
33 </body>
34 </html>
```

